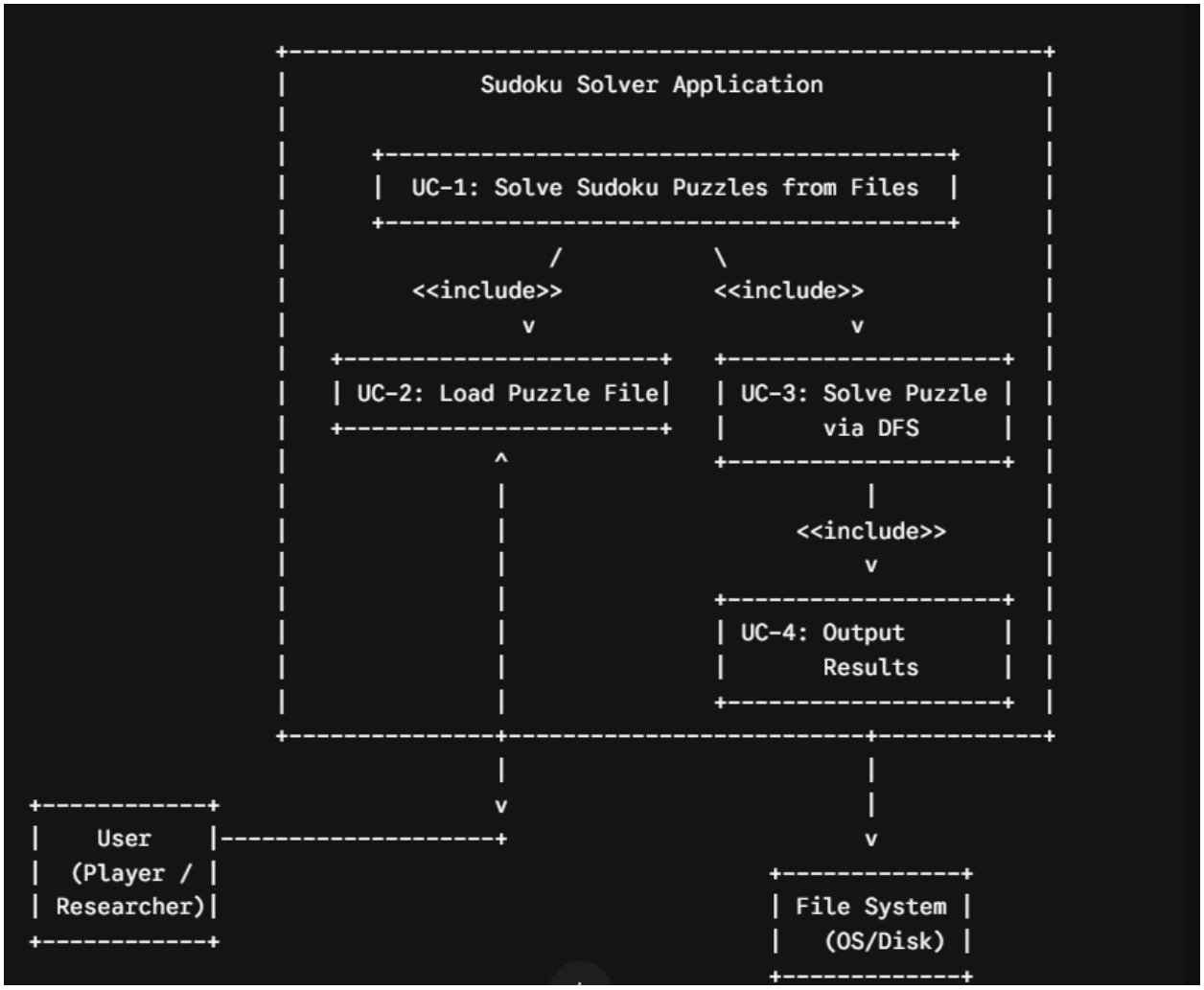



```
@startuml
left to right direction
skinparam packageStyle rectangle

actor "User" as User
actor "File System" as FS <<Secondary>>

rectangle "Sudoku Solver System" {
   usecase "UC-1: Solve Sudoku Puzzles from Files" as UC1
   usecase "UC-2: Load Puzzle File" as UC2
   usecase "UC-3: Solve Puzzle via Backtracking" as UC3
   usecase "UC-4: Output Results & Solutions" as UC4
   usecase "UC-5: Display 'No Solution Found'" as UC5

   User --> UC1
   UC1 .> UC2 : <<include>>
   UC1 .> UC3 : <<include>>
```

```
    UC3 .> UC4 : <<include>>
    UC3 <. UC5 : <<extend>>
    UC2 --> FS
}
@enduml
```

a. Identified Use Cases

- **Primary Use Case:**
  - **UC-1: Solve Sudoku Puzzles from Files** (Core business goal of running batch puzzle executions).
- **Supporting / Included Use Cases:**
  - **UC-2: Load Puzzle File** (Reading and parsing 9x9 grid data from local text files puzzle1.txt through puzzle5.txt).
  - **UC-3: Solve Puzzle via Backtracking** (Applying depth-first search to find all valid candidate values for empty cells).
  - **UC-4: Output Results & Solutions** (Printing file name, initial board, and all discovered completed solutions).
  - **UC-5: Report No Solution** (Handling unsolvable boards).

b. Brief Description

UC-1: Solve Sudoku Puzzles from Files

The User launches the solver application to process a set of Sudoku puzzle files. The system sequentially loads each puzzle file from disk, parses the initial clues into a 9x9 matrix, displays the original board, uses recursive depth-first search with backtracking to exhaustively explore all valid digit placements, and outputs every valid solution found or indicates that no solution exists.

c. Step-by-Step Flow (Outline) of Events

1. **Initiation:** The User executes the program.
2. **File Selection:** The system iterates through the target puzzle files (puzzle1.txt – puzzle5.txt).
3. **Data Loading:** For each file, the system reads the 81 grid cells, mapping digits 1–9 and underscores _ into the board object.
4. **Initial Display:** The system prints the file name and the initial 9x9 board layout.
5. **Candidate Search & Backtracking:** The system recursively traverses unfilled cells:
   - Identifies candidate values (1–9) conforming to row, column, and 3x3 block constraints.
   - Places candidate values tentatively and recurses deeper.
   - Backtracks upon hitting invalid configurations.
6. **Solution Emission:** Each time all 81 cells are legally filled, the system outputs that specific completed grid and increments the solution counter.
7. **Completion / Fallback:** If zero solutions are discovered after exhaustive search, the system prints "No solution found".

8. **Loop:** Steps 3–7 repeat for all remaining puzzle files until all target puzzles are processed.

d. Detailed Use Case Description

Attribute	**Specification**
Use Case ID	**UC-1**
Use Case Name	Solve Sudoku Puzzles from Files
Primary Actor	User / Student / Evaluator
Secondary Actor	File System (OS)
Trigger	User runs the Sudoku Solver executable from the command line or IDE.
Preconditions	1. The executable is compiled and operational. 2. Target files (puzzle1.txt to puzzle5.txt) reside in the expected working directory. 3. File reading permissions are granted.

Postconditions	1. All puzzle files are read and processed. 2. For every file, the initial puzzle and either all valid complete solutions or "No solution found" are displayed on standard output.

Main Success Scenario (Basic Flow)

1. **User** initiates execution of the Sudoku solver program.
2. **System** identifies the batch list of target puzzle files (puzzle1.txt through puzzle5.txt).
3. For each puzzle file in the list:
    1. **System** requests the file stream from the **File System**.
    2. **File System** delivers the contents of the file.
    3. **System** parses the 81 characters into a 9x9 grid object, identifying clues (1–9) and empty cells (_).
    4. **System** outputs the file header and renders the starting grid to stdout.
    5. **System** initiates the depth-first backtracking search starting at cell (0, 0).
    6. **System** finds an empty cell, inspects intersecting rows, columns, and 3x3 blocks, and generates a valid candidate list.
    7. **System** assigns the first candidate value and recursively calls the search on the next cell.
    8. Upon filling all 81 cells, **System** displays the formatted solution and resumes backtracking to discover alternative solutions.
    9. **System** outputs the total count of solutions found for that file.
4. **System** terminates successfully once the file list is exhausted.

Alternative Flows

- **Alt 1 (Multiple Solutions Found):**
    - At step 3.8, if a puzzle allows multiple permutations, the system prints each valid grid as it is completed (Solution #1:, Solution #2:, etc.) and outputs the cumulative count.
- **Alt 2 (No Solution Exists):**
    - At step 3.8, if the backtracking tree terminates and the solution counter equals 0, the system displays: "No solution found".

Exception Flows

- **Exc 1 (File Not Found / Unreadable):**
    - At step 3.1, if the target file cannot be opened, the system outputs an error message ("Error: Could not open file <filename>"), skips the file, and proceeds to the next puzzle in the queue.

- **Exc 2 (Malformed Puzzle Layout):**
    - At step 3.3, if a file contains fewer than 81 recognizable cell entries or illegal characters, the system reports an invalid format error and skips solving that file.

Special Requirements

- **Determinism:** The search must be exhaustive so all possible valid solutions are captured.
- **State Encapsulation:** The internal grid matrix and solving algorithm must be self-contained within an object, avoiding global state.